# NEIGHBORHOOD ACTION DASHBOARD

Relatório técnico | Aplicação de três regras de decisão lógica

INTEGRANTES

**Guilherme Dias** RM 576403

**Matheus Nunes** RM 574511

## 01 OBJETIVO

O projeto simula demandas de um bairro e aplica três decisões booleanas a cada ocorrência: prioridade, pendência cadastral e necessidade de acompanhamento. As entradas são coletadas durante o cadastro ou atualizadas posteriormente.

## 02 CRUD COMO BASE DA APLICAÇÃO

| | |
|---|---|
| **CREATE** | Cadastrar ocorrência e suas características. |
| **READ** | Consultar ocorrências e verificar os resultados das regras. |
| **UPDATE** | Atualizar status, cadastro completo e localização. |
| **DELETE** | Remover registros incorretos ou duplicados. |

## 03 NOTAÇÃO UTILIZADA

Entradas: A e B | Saída: s | 0 = falso/não | 1 = verdadeiro/sim

```
AND = *      OR = +      NOT = '      NOR = (A + B)'
```

## 04 ESCOPO E MATERIAL DA EQUIPE

Protótipo em Java, com armazenamento temporário em memória e sem integração com órgãos públicos. As páginas seguintes apresentam, para cada regra, a expressão booleana, os significados binários e as capturas de tela produzidas pela equipe (tabela-verdade, circuito e trecho do Java).

Critério central: demonstrar correspondência entre o problema descrito, os valores de A e B, a expressão, o circuito e o resultado da implementação.

# REGRA 1 | IDENTIFICAÇÃO DE PRIORIDADE

Uma ocorrência é prioritária somente quando é urgente e afeta a coletividade.

## ENTRADAS E SAÍDA BINÁRIAS

| Variável | 0 = falso / não | 1 = verdadeiro / sim |
|---|---|---|
| A | Não é urgente | É urgente |
| B | Não afeta a coletividade | Afeta a coletividade |
| s | Não é prioritária | É prioritária |

**EXPRESSÃO BOOLEANA** $s = A * B$

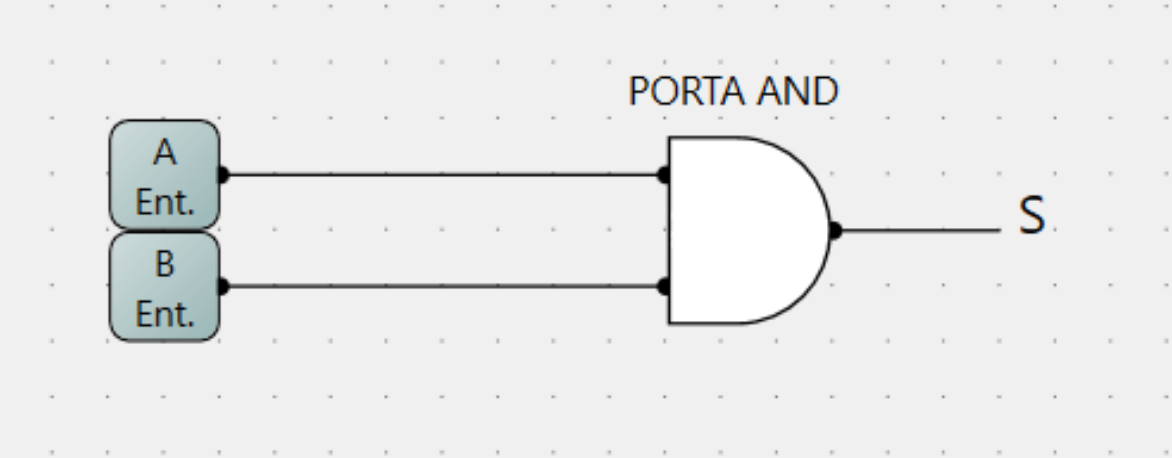
## TABELA-VERDADE | CAPTURA DA EQUIPE

*Tabela-Verdade*

| A | B | S |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

AND: a saída vale 1 somente se A = 1 e B = 1.

## CIRCUITO LÓGICO | CAPTURA DA EQUIPE




## TRECHO DO CÓDIGO JAVA | CAPTURA DA EQUIPE

```java
// Regra 1 - AND: s = A * B (urgente E coletivo).
static boolean prioridade(boolean A, boolean B) { return A && B; }
```

As três representações são compatíveis com o método Java correspondente.

# REGRA 2 | IDENTIFICAÇÃO DE PENDÊNCIA CADASTRAL

Existe pendência quando o cadastro está incompleto ou a localização não foi informada.

## ENTRADAS E SAÍDA BINÁRIAS

| Variável | 0 = falso / não | 1 = verdadeiro / sim |
|---|---|---|
| A | Cadastro incompleto | Cadastro completo |
| B | Localização ausente | Localização informada |
| s | Sem pendência | Com pendência |

**EXPRESSÃO BOOLEANA** $s = A' + B'$

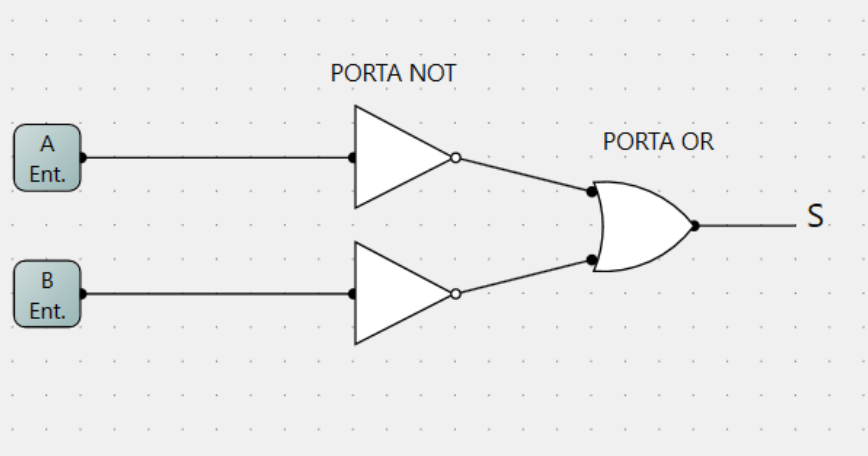
## TABELA-VERDADE | CAPTURA DA EQUIPE

*Tabela-Verdade*

| A | B | S |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

NOT + OR: A e B são invertidas; basta uma ausência para a saída valer 1.

## CIRCUITO LÓGICO | CAPTURA DA EQUIPE




## TRECHO DO CÓDIGO JAVA | CAPTURA DA EQUIPE

```java
// Regra 2 - NOT + OR: s = A' + B' (cadastro OU localizacao ausente).
static boolean pendencia(boolean A, boolean B) { return !A || !B; }
```

As três representações são compatíveis com o método Java correspondente.

# REGRA 3 | NECESSIDADE DE ACOMPANHAMENTO

A ocorrência precisa de acompanhamento enquanto não estiver concluída nem cancelada.

## ENTRADAS E SAÍDA BINÁRIAS

| Variável | 0 = falso / não | 1 = verdadeiro / sim |
|---|---|---|
| A | Não está concluída | Está concluída |
| B | Não está cancelada | Está cancelada |
| s | Sem acompanhamento | Com acompanhamento |

**EXPRESSÃO BOOLEANA** $s = (A + B)'$

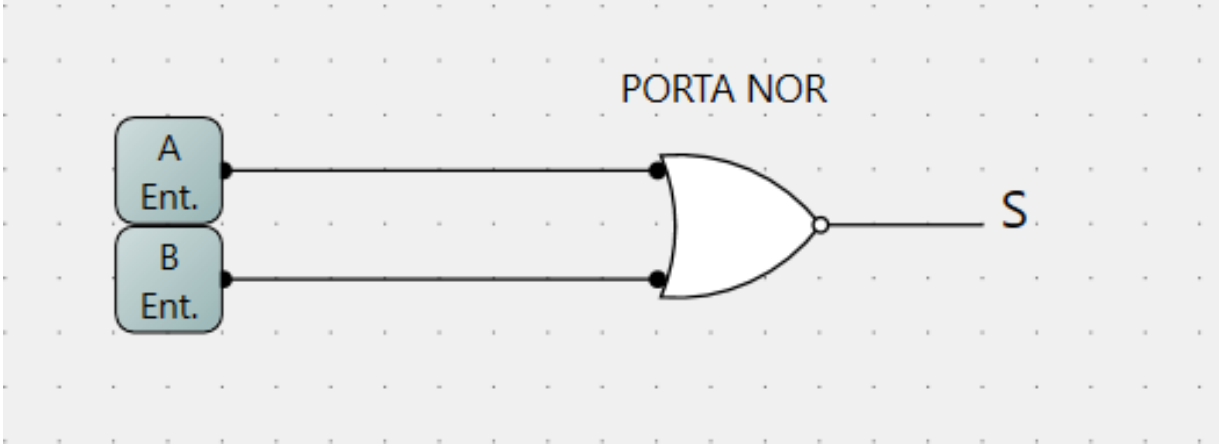
## TABELA-VERDADE | CAPTURA DA EQUIPE

*Tabela-Verdade*

| A | B | S |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

NOR: a saída vale 1 somente quando A = 0 e B = 0.

## CIRCUITO LÓGICO | CAPTURA DA EQUIPE




## TRECHO DO CÓDIGO JAVA | CAPTURA DA EQUIPE

```java
// Regra 3 - NOR: s = (A + B)' (nao concluida NEM cancelada).
static boolean acompanhamento(boolean A, boolean B) { return !(A || B); }
```

Nota: A = 1 e B = 1 é combinação teórica; um único status não pode ser ambos.

# TESTES E RESULTADOS

As funções foram testadas com todas as combinações binárias das entradas A e B. Os resultados das três saídas foram comparados com os valores esperados das tabelas-verdade.

### 05 EVIDÊNCIA DE EXECUÇÃO DO JAVA (OPÇÃO 6)

```
A B | R1 R2 R3 | Testes
0 0 |  0  1  1 | 3/3 OK
0 1 |  0  1  0 | 3/3 OK
1 0 |  0  1  0 | 3/3 OK
1 1 |  1  0  0 | 3/3 OK
Total: 12/12 testes corretos.
```

### 06 EXEMPLO PRÁTICO COM UMA OCORRÊNCIA SIMULADA

Cadastro: ID 1; tipo saneamento; descrição "bueiro vazando"; urgente = 1; afeta a coletividade = 1; cadastro completo = 1; localização inicialmente vazia.

| | |
|---|---|
| **Antes da atualização** | Prioridade 1 \| Pendência 1 \| Acompanhamento 1 |
| **Localização informada** | Rua das Flores, 25 |
| **Depois da atualização** | Prioridade 1 \| Pendência 0 \| Acompanhamento 1 |

### 07 INSTRUÇÕES PARA EXECUTAR

```
javac -encoding UTF-8 NeighborhoodActionDashboard.java
java NeighborhoodActionDashboard
```

O menu oferece cadastro, consulta, atualização, exclusão, indicadores e testes. O arquivo NeighborhoodActionDashboard.java acompanha o relatório em separado, no mesmo ZIP.

### CONCLUSÃO

O CRUD fornece e atualiza os dados de cada ocorrência. O objetivo do trabalho é mostrar a aplicação das portas AND, OR, NOT e NOR por meio de três regras, com correspondência entre expressões, tabelas-verdade, circuitos e implementação.